

Pemrograman Web Lanjut

SIB245007 | D-IV Sistem Informasi Bisnis

Pertemuan 7: **Autentikasi, Otorisasi, dan RBAC**

Siapa kamu, apa yang boleh kamu lakukan, dan bagaimana memeriksanya di setiap request

Yang Akan Kamu Pelajari

1. Membedakan **autentikasi** (membuktikan identitas) dan **otorisasi** (menentukan izin), serta mengapa keduanya perlu dipisah
2. Memahami bagaimana **sesi**, **cookie**, dan fitur terkait (remember me, lupa kata sandi, pembatasan percobaan login) membuat login terasa aman dan nyaman
3. Mengenali aturan dasar penyimpanan **kata sandi** dan **RBAC** (kontrol akses berbasis peran) lewat **middleware**, **Gate**, dan **Policy**

Slide ini membahas konsep. Implementasi login dan pembatasan peran pada Simple POS dikerjakan di jobsheet praktikum.

Bagian 1

Autentikasi: Membuktikan Identitas

Dua Pertanyaan yang Berbeda

Autentikasi

- Pertanyaannya: "kamu siapa?"
- Dibuktikan lewat email dan kata sandi
- Hasilnya: identitas pengguna diketahui

Otorisasi

- Pertanyaannya: "kamu boleh apa?"
- Dicek setelah identitas diketahui
- Hasilnya: halaman atau aksi tertentu diizinkan atau ditolak

Pegawai yang kartunya sah di lobi tetap bisa ditolak di ruang server. Lobi mengecek identitas, ruang server mengecek izin.

Mengapa HTTP Butuh Sesi

Stateless: setiap request HTTP berdiri sendiri. Server tidak otomatis ingat request sebelumnya, walaupun dari browser yang sama.

- Tanpa mekanisme tambahan, setiap halaman terlihat seperti dibuka oleh orang asing
- Sesi memberi server cara mengingat pengguna di antara request, tanpa meminta login ulang di setiap halaman

Alur Login dengan Sesi



- Request berikutnya membawa cookie yang sama, server mencari sesinya, lalu tahu siapa pengguna itu
- Cookie hanya berisi ID sesi, data penggunaanya tetap tersimpan di server

Cookie dan ID Sesi

Sesi: data pengguna yang login, disimpan di server dan diidentifikasi oleh satu ID acak yang dikirim lewat cookie.

- ID sesi harus sulit ditebak, karena siapa pun yang memegangnya bisa berpura-pura menjadi pengguna itu
- Cookie yang sensitif sebaiknya diberi flag `HttpOnly` (tidak bisa dibaca JavaScript) dan `Secure` (hanya lewat HTTPS)

Remember Me: Login yang Bertahan

Remember Me: cookie tambahan berumur panjang (berminggu-minggu) yang membuat pengguna tetap login walau sesinya di server sudah berakhir, misalnya setelah menutup browser.

- Cookie ini membawa token acak terpisah, bukan sesi biasa dan bukan kata sandi, disimpan di kolom seperti `remember_token` pada tabel pengguna
- Begitu pengguna kembali, server mencocokkan token itu, membuat sesi baru, lalu menggantinya dengan token baru lagi
- Karena umurnya panjang, centang "Remember Me" sebaiknya jadi pilihan pengguna sendiri, bukan perilaku default

Session vs Token

Sesi (server menyimpan)

- Server menyimpan status login
- Cabut akses cukup hapus sesinya
- Cocok untuk aplikasi web dengan halaman di browser

Token (klien menyimpan)

- Klien membawa bukti identitas yang ditandatangani
- Server tidak perlu menyimpan status login
- Cocok untuk API yang dipakai banyak klien, dibahas lebih lanjut nanti

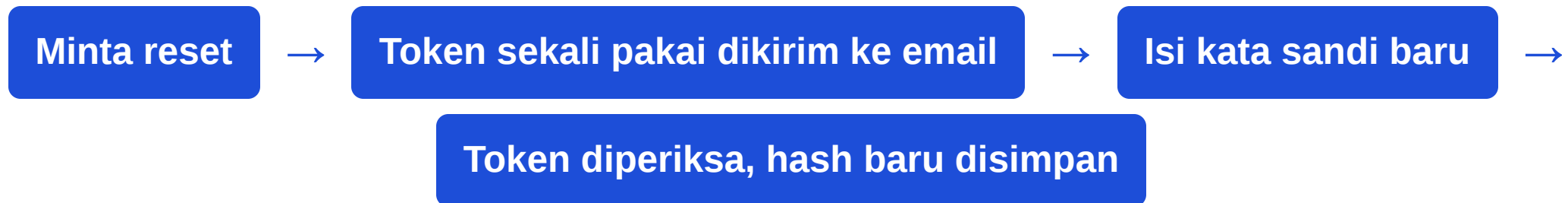
Kata Sandi Tidak Pernah Disimpan Mentah

Hash: hasil fungsi satu arah dari kata sandi, ditambah salt acak, sehingga kata sandi asli tidak bisa dibaca kembali dari basis data.

- Kalau basis data bocor, penyerang hanya mendapat hash, bukan kata sandi yang bisa langsung dipakai
- Saat login, server menghitung hash dari kata sandi yang diketik lalu membandingkannya dengan hash tersimpan

Lupa Kata Sandi

Karena kata sandi disimpan sebagai hash, server sendiri tidak bisa membacanya untuk dikirim ulang lewat email. Solusinya bukan mengirim kata sandi lama, melainkan memberi kesempatan membuat kata sandi baru:



- Token reset harus kedaluwarsa dalam waktu singkat dan hanya bisa dipakai sekali, persis seperti ID sesi yang diregenerasi setelah login
- Link reset dikirim lewat email, bukan ditampilkan langsung di halaman, supaya hanya pemilik email asli yang bisa memakainya

Mengapa Hashing Sengaja Dibuat Lambat

- Hashing yang cepat membuat penyerang bisa mencoba miliaran tebakan per detik
- Algoritma password modern (bcrypt, argon2) sengaja dibuat lambat dan bisa dinaikkan biayanya seiring perangkat makin cepat
- Di Laravel, `Hash::make()` memakai algoritma yang diatur di konfigurasi, dan jangan pernah menulis fungsi hash sendiri

Membatasi Percobaan Login

Hashing yang lambat memperlambat penyerang yang punya salinan basis data. Tapi penyerang yang hanya menebak lewat form login butuh pertahanan lain: membatasi berapa kali percobaan login boleh dilakukan dalam waktu tertentu.

```
Route::post('/login', [LoginController::class, 'store'])  
    ->middleware('throttle:5,1');
```

- Kode di atas membatasi maksimal 5 percobaan per menit dari sumber yang sama, percobaan ke-6 langsung ditolak tanpa mengecek kata sandinya
- Sama seperti hashing yang lambat, tujuannya membuat serangan coba-coba makan waktu sangat lama, bukan mencegahnya sepenuhnya

Session Fixation

Serangan session fixation: penyerang memberi korban ID sesi yang sudah dia ketahui sebelum login. Begitu korban login, sesi yang sama ikut menjadi milik penyerang.

- Cara mencegahnya: buat ID sesi baru tepat setelah login berhasil, lalu hapus ID lama
- Di Laravel, ini dilakukan dengan `session()->regenerate()` setelah `Auth::attempt()` berhasil

Bagian 2

Otorisasi dan RBAC

Otorisasi: Izin Setelah Identitas Terbukti

Otorisasi: keputusan apakah pengguna yang sudah terautentikasi boleh melakukan satu aksi tertentu pada satu sumber daya tertentu.

- Identitas yang benar belum berarti izin yang benar
- Contoh: editor boleh mengedit artikel, pembaca biasa hanya boleh membacanya

Peran, Bukan Pengguna Satu per Satu



RBAC (Role-Based Access Control): izin dikaitkan ke peran, lalu pengguna mendapat izin lewat peran yang dimilikinya.

- Menambah pengguna baru cukup memberinya peran, tidak perlu mengatur izin satu per satu

Middleware sebagai Pos Pemeriksaan



- Middleware memeriksa request sebelum sampai ke controller, dipasang pada route atau grup route
- Urutannya penting: pastikan sudah login dulu, baru memeriksa peran

401 vs 403

401 Unauthorized

- Belum ada identitas yang terbukti
- Solusinya: login dulu
- Di browser, Laravel menjawabnya dengan mengarahkan ke halaman login

403 Forbidden

- Identitas sudah diketahui, tetapi tidak punya izin
- Solusinya: bukan login ulang, melainkan izin yang berbeda

Route yang Lupa Dilindungi

Middleware hanya melindungi route yang benar-benar memakainya. Route admin baru yang ditambahkan di luar grup yang dilindungi bisa diakses siapa pun yang sudah login, tanpa peduli perannya.

- Kesalahan ini tidak terlihat saat pengujian, kalau yang mengecek selalu login sebagai admin
- Cara mencegahnya: uji juga dengan akun peran lain, bukan hanya akun admin

Gate: Aturan Otorisasi Sederhana

Gate: aturan otorisasi berbentuk closure, didaftarkan dengan nama, dan dipanggil lewat nama itu di mana pun dibutuhkan.

```
Gate::define('kelola-produk', function (User $user) {  
    return $user->role === 'admin';  
});  
  
Gate::authorize('kelola-produk');
```

- Cocok untuk aturan yang tidak terikat ke satu baris data tertentu, misalnya "siapa boleh membuka halaman pengaturan"
- Middleware memeriksa di level route, Gate bisa dipanggil di mana saja di dalam kode, termasuk di tengah controller

Policy: Otorisasi per Model

Policy: class yang mengumpulkan semua aturan otorisasi untuk satu model, satu method per aksi (view, update, delete, dst.).

```
class ArticlePolicy
{
    public function update(User $user, Article $article): bool
    {
        return $user->id === $article->author_id;
    }
}
```

- Dibuat lewat `php artisan make:policy ArticlePolicy --model=Article`, lalu dipanggil dengan `Gate::authorize('update', $article)`
- Method menerima baris data yang sebenarnya, jadi aturannya bisa berbeda untuk tiap baris, bukan cuma berdasarkan peran

Middleware vs Policy: Kapan Pakai Apa

Middleware

- Pertanyaannya: "boleh masuk ke route ini?"
- Diperiksa sebelum controller, untuk seluruh request
- Cocok untuk aturan berbasis peran, sama untuk semua baris data

Policy

- Pertanyaannya: "boleh melakukan ini pada baris data ini?"
- Diperiksa di dalam controller, untuk satu baris data tertentu
- Cocok untuk aturan kepemilikan, misalnya "hanya penulis asli yang boleh mengedit"

Middleware `role:admin` yang kamu buat di jobsheet menjawab pertanyaan pertama. Begitu aplikasi kasir butuh aturan seperti "kasir hanya boleh membatalkan transaksinya sendiri", itu pertanyaan kedua, pekerjaan Policy.

Memeriksa Izin di Blade: @can dan @cannot

```
@can('update', $article)
    <a href="{{ route('articles.edit', $article) }}">Edit</a>
@endcan

@cannot('update', $article)
    <p>Kamu tidak bisa mengedit artikel ini.</p>
@endcannot
```

- @can / @cannot memanggil Gate atau Policy yang sama, jadi aturannya tidak pernah ditulis dua kali
- Sama seperti @if (auth()->user()->isAdmin()) yang menyembunyikan menu admin di navigasi, @can hanya menyembunyikan tampilan, pemeriksaan sungguhan tetap wajib dilakukan lagi di controller atau route

Bagian 3

Pilihan Paket dan Perbandingan Lintas Framework

Tiga Pendekatan di Laravel

Pendekatan	Kompleksitas setup	Cocok untuk
Middleware kustom	Rendah	Peran sederhana dan tetap, misalnya 2 sampai 3 peran
Laravel Breeze	Sedang	Autentikasi siap pakai beserta tampilan login dan register
Spatie Permission	Sedang sampai tinggi	Izin sangat rinci per aksi dengan banyak kombinasi peran

Dokumentasi resmi: laravel.com/docs/authentication

Kapan Memilih Apa

- Middleware kustom cukup selama peran sedikit dan jarang berubah
- Breeze menghemat waktu membangun halaman login dan register, tetapi tetap perlu dipahami isinya
- Spatie Permission masuk akal saat izin harus dibagi per cabang, per data, atau per aksi
- Mulai dari yang paling sederhana, lalu naik ke paket yang lebih besar saat kebutuhan nyata muncul

Perbandingan Lintas Framework

Framework	Autentikasi bawaan	Pendekatan umum untuk otorisasi
Laravel	Auth, session, middleware	Middleware, Gate/Policy, paket seperti Spatie
Django	Auth dengan user, group, dan permission	Permission per model dan group
Express (Node.js)	Tidak ada, memakai paket	Passport.js atau JWT, middleware kustom
Next.js	Tidak ada, memakai paket	Library seperti Auth.js (NextAuth)

Rangkuman (1/3)

- Autentikasi menjawab "kamu siapa", otorisasi menjawab "kamu boleh apa"; keduanya dicek terpisah
- Sesi dan cookie membuat HTTP yang stateless bisa mengingat pengguna; cookie hanya membawa ID sesi, remember me memakai cookie terpisah berumur panjang
- Kata sandi disimpan sebagai hash, dan setelah login sesi harus diperbarui (regenerate) untuk mencegah session fixation

Rangkuman (2/3)

- Lupa kata sandi diselesaikan lewat token reset sekali pakai yang dikirim ke email, bukan mengirim ulang kata sandi lama
- Membatasi percobaan login (throttle) memperlambat tebakan brute-force, sama seperti hashing yang sengaja dibuat lambat
- RBAC mengaitkan izin ke peran, dan middleware menjadi pos pemeriksaan di level route sebelum controller

Rangkuman (3/3)

- Gunakan 401 untuk belum login dan 403 untuk sudah login tetapi tidak berwenang
- Gate menjawab aturan sederhana yang tidak terikat satu baris data, Policy menjawab aturan per baris data (misalnya kepemilikan); `@can` / `@cannot` di Blade memanggil aturan yang sama
- Pilih paket sesuai kebutuhan: middleware kustom untuk peran sederhana, Breeze untuk autentikasi siap pakai, Spatie untuk izin yang sangat rinci

Referensi & Diskusi

Dokumentasi resmi Laravel (Authentication, Authorization, Middleware)

Kode lengkap: `github.com/se-polinema/simple-pos-ch07`

Pertemuan berikutnya: UTS (Evaluasi Progres Proyek PBL)